Creating PsotgreSQL and PGADMIN Services with Docker

1. Goal of this lecture

- Create two services using Docker
  - PostgreSQL
  - PGADMIN
- Use a **Docker Networks** so both containers can communicate.
- Use **Docker Volumes** so database/admin data persists
- Configure containers to **restart automatically** unless explicitly stopped

2. Create Docker Network

Command:

```bash
docker network create postgres
```

- Creates a Docker network named:

```
postgres
```

- Both PostgreSQL and pgAdmin containers will use this network.
- This allows the containers to **communicate with each other**.

Pasted text

3. PostgreSQL Container

Bash

```
docker run -d \
  --name postgres-container \
  -e POSTGRES_USER=<username> \
  -e POSTGRES_PASSWORD=<password> \
  -e PGDATA=/data/postgres \
  -v postgres:/data/postgres \
  -p 5432:5432 \
  --network postgres \
  --restart unless-stopped \
  postgres
```

Let's understand it piece by piece.

4. –d Detached Mode

```
Bash

-d
```

- Runs the container in **detached/background mode**.
- Your terminal remains available for other commands.

5. --name

Bash

```
--name postgres-container
```

- Gives the PostgreSQL container a specific name.
- Here the container name is:

```
postgres-container
```

- Having a meaningful name makes it easier to reference the container in Docker commands.

6. PostgreSQL Environment Variables

Username

```bash
-e POSTGRES_USER=<username>
```

- Sets the PostgreSQL database username.

Password

```bash
-e POSTGRES_PASSWORD=<password>
```

- Sets the password for the PostgreSQL user.

PGDATA

```bash
-e PGDATA=/data/postgres
```

- Specifies the PostgreSQL data directory inside the container:

```
/data/postgres
```

7. Docker Volume

The command contains:

```bash
-v postgres:/data/postgres
```

This mounts a Docker volume named:

```
postgres
```

to:

```
/data/postgres
```

inside the container.

Why?

To **persist PostgreSQL data**.

Without persistent storage, removing a container can cause you to lose the data stored inside that container.

The basic idea is:

```
Docker Volume
     ↓
PostgreSQL data
     ↓
Survives container recreation
```

Remember

```
-v volume-name:container-path
```

8. PostgreSQL PORT Mapping

Bash

```
-p 5432:5432
```

This maps:

```
HOST                    CONTAINER
5432        ──────→     5432
```

So PostgreSQL can be accessed through the host's port `5432`.

9. Connect PostgreSQL to Docker Network

Bash

```
--network postgres
```

- Connects the PostgreSQL container to the Docker network named `postgres`.
- This is important because pgAdmin will also be connected to the **same network**.

10. - - restart unles-stopped

Bash

```
--restart unless-stopped
```

This tells Docker:

> Automatically restart the container unless I explicitly stop it.

This is useful for services such as:

- PostgreSQL
- pgAdmin
- Other infrastructure services

11. PostgreSQL Image

Finally:

```bash
postgres
```

- This is the Docker image used to create the PostgreSQL container.
- Docker will:
  1. Check whether the image exists locally.
  2. If it doesn't, pull it from the registry.
  3. Create the container.
  4. Start it.

12. pgAdmin Container

Conceptually:

Bash

```
docker run -d \
  --name pgadmin \
  -e PGADMIN_DEFAULT_EMAIL=<email> \
  -e PGADMIN_DEFAULT_PASSWORD=<password> \
  -e PGADMIN_CONFIG_SERVER_MODE=False \
  -v pgadmin:/var/lib/pgadmin \
  -p 5050:80 \
  --network postgres \
  --restart unless-stopped \
  dpage/pgadmin4
```

13. pgAdmin Environment Variables

Default Email

```Bash
-e PGADMIN_DEFAULT_EMAIL=<email>
```

- Sets the default email/login for pgAdmin.

Default Password

```Bash
-e PGADMIN_DEFAULT_PASSWORD=<password>
```

- Sets the default pgAdmin password.

Server Mode

```Bash
-e PGADMIN_CONFIG_SERVER_MODE=False
```

- The lecture explains this as disabling server mode for pgAdmin.

14. pgAdmin volume

Bash

```
-v pgadmin:/var/lib/pgadmin
```

- Creates/mounts a Docker volume named:

```
pgadmin
```

- It is mounted to the pgAdmin data directory inside the container.
- This is used to **persist pgAdmin data**.

So we now have two volumes:

```
postgres volume
      ↓
PostgreSQL data

pgadmin volume
      ↓
pgAdmin data
```

15. pgAdmin Port Mapping

Bash

```
-p 5050:80
```

This means:

```
HOST                    CONTAINER
5050        ──────→      80
```

Therefore, the pgAdmin web interface can be accessed through:

```
localhost:5050
```

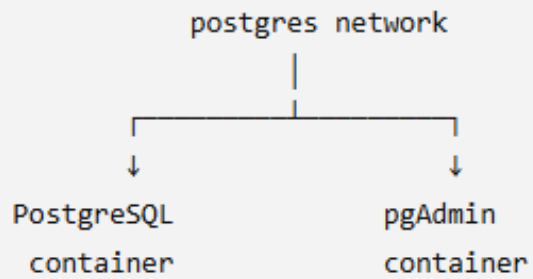
The container itself listens on port `80`.

16. pgAdmin uses the same network

Bash

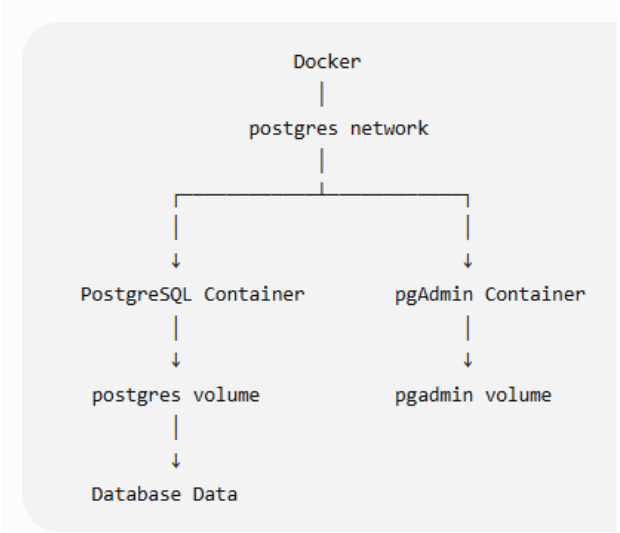
```
--network postgres
```

Both containers now belong to:



Therefore they can communicate using their container names.

17. Complete Architecture



And from your Windows machine:

```
localhost:5432
      ↓
PostgreSQL container

localhost:5050
      ↓
pgAdmin container
```

18. Important Concepts

Option	Meaning
`docker run`	Create and start container
`-d`	Detached/background mode
`--name`	Give container a name
`-e`	Set environment variable
`-v`	Mount Docker volume
`-p`	Map host port → container port
`--network`	Connect container to Docker network
`--restart unless-stopped`	Automatically restart unless explicitly stopped
`postgres`	PostgreSQL Docker image
`dpage/pgadmin4`	pgAdmin Docker image

19. The Big Picture

This lecture combines **three Docker concepts** you've learned separately:

